FastAPI 기본 문법 이해

HAIC 모델 서빙 실습 사전 준비

모델 서빙 및 AIOps 수업 실습 전, project/ 코드에 실제로 등장하는 모델 서빙 내용 위주 설명입니다.

- 이 문서는 FastAPI 를 처음 접하는 실습생을 위한 자료로, 일반적인 FastAPI 전체 문법이 아니라 project/ 스켈레톤 코드에 실제로 등장하는 문법만 다룹니다.
- 순서는 ① 엔드포인트가 어떻게 조립되는가(구조) → ② 그 구조를 채우는 데 필요한 문법 6 가지입니다. 구조를 먼저 알아야 각 문법이 코드의 어느 자리에 들어가는지 헷갈리지 않습니다.
- 각 절의 표는 "문법 | 이 프로젝트에서의 실제 예시" 순서로, 실습 코드를 그대로 인용합니다.

엔드포인트 구성 구조

이 프로젝트의 서빙 서버(serving_app/)는 하나의 FastAPI 앱에 기능별 라우터를 조립하는 구조입니다. 아래 순서로 해부하면, 어떤 코드를 보더라도 "지금 이 부분이 전체 구조의 어디에 해당하는지" 바로 알 수 있습니다.

순서	구성 요소	코드 위치	역할
1	`FastAPI()` 앱 인스턴스	`main.py`	서버 전체를 대표하는 객체. 여기에 모든 라우터가 모입니다.
2	`APIRouter(prefix=...)`	`routers/*.py`	기능별로 엔드포인트를 묶는 단위 (`data.py`→/data, `logs.py`→/logs)
3	`@router.get()`/`@router.post()` + 핸들러 함수	`routers/*.py`	HTTP 메서드 + 경로를 함수에 매핑. 함수가 요청을 실제로 처리합니다.
4	Pydantic 스키마(요청/응답)	`schemas.py`	핸들러가 받는 입력과 돌려주는 출력의 모양을 정의·검증합니다.
5	`app.include_router()`	`main.py`	라우터를 앱에 연결합니다 (등록 순서가 경로 우선순위를 결정).
6	`app.mount("/", StaticFiles(...))`	`main.py`	대시보드 HTML을 같은 서버에서 서빙 (반드시 라우터 등록 뒤에).

요청 한 건이 처리되는 순서도 이 표와 같습니다: URL이 라우터에 매칭 → Pydantic이 입력을 검증 → 핸들러 함수 실행 → return한 값이 자동으로 JSON 응답이 됩니다. 아래 6개 절은 이 순서를 채우는 문법을 하나씩 다룹니다.

이 프로젝트의 엔드포인트

Method	URL	파일
GET	`/`	main.py (StaticFiles - 대시보드)
POST	`/predict`	`routers/predict.py`
GET	`/health`	`routers/health.py`
POST	`/data/upload`	`routers/data.py`
GET	`/data/status`	`routers/data.py`
POST	`/predict/batch-test`	`routers/predict.py`
GET	`/logs`, `/logs/{filename}`	`routers/logs.py`

1. 앱/라우터 구조

앱 인스턴스를 만들고, 기능별로 라우터를 나눈 뒤 조립하는 패턴입니다. 데코레이터가 HTTP 메서드와 URL을 함수에 매핑한다는 것, 그리고 def와 async def의 차이만 알면 이 프로젝트의 라우터 코드는 전부 읽힙니다.

문법	이 프로젝트에서의 예시
`FastAPI(title="...")`	main.py: `app = FastAPI(title="HAIC Serving & AIOps")`
`APIRouter(prefix="/data")`	routers/data.py: `router = APIRouter(prefix="/data")`
`@router.get("/health")`	routers/health.py
def vs async def	routers/data.py의 `upload()`만 `async def` - 나머지는 전부 `def`

> *async def는 함수 안에서 await로 I/O를 기다려야 할 때만 씁니다. 이 프로젝트에서는 업로드 파일을 읽는 upload() 하나뿐이고, 나머지는 전부 평범한 def입니다.*

2. 요청을 받는 3가지 방법

이 프로젝트에 실제로 쓰인 것만 있습니다. 쿼리 파라미터는 쓰이지 않으므로 다루지 않습니다.

2-1. 경로 파라미터

URL 경로의 `{filename}` 부분을 함수 인자로 그대로 받습니다. 중괄호 안 이름과 함수 인자 이름이 같아야 자동으로 연결됩니다.

```
@router.get("/{filename}")
def read_log(filename: str): ...
```
(routers/logs.py)

2-2. 요청 바디

Pydantic 모델을 함수 인자의 타입으로 선언하면, FastAPI가 요청 본문(JSON)을 자동으로 파싱·검증해 넘겨줍니다.

```
def predict(req: PredictRequest): ...
```
(routers/predict.py)

2-3. 파일 업로드

UploadFile과 File(...)로 받고, await file.read()로 내용을 읽습니다 (이 프로젝트에서 유일한 async 핸들러).

```
async def upload(file: UploadFile = File(...)):
    raw = await file.read()
```
(routers/data.py)

3. Pydantic으로 스키마 정의

요청/응답의 모양을 클래스로 선언하면, FastAPI가 자동으로 검증하고 실패 시 422 응답을 만들어 줍니다.

문법	이 프로젝트에서의 예시
`class X(BaseModel): field: type`	schemas.py: `DailyPoint(close: float, volume: int)`
`Field(..., gt=0, ge=0, min_length=, max_length=)`	schemas.py의 `PredictRequest.sequence` (길이 정확히 20)
중첩 모델 `list[DailyPoint]`	schemas.py의 `PredictRequest.sequence`
`response_model=PredictResponse`	routers/predict.py: `@router.post("/predict", response_model=PredictResponse)`

> *422 Unprocessable Entity가 뜬다는 것은 스키마 검증이 "의도대로" 동작했다는 뜻입니다. 응답의 loc·msg를 읽으면 어떤 필드가 왜 걸렸는지 알 수 있습니다.*

4. 응답과 에러 처리

문법	이 프로젝트에서의 예시
dict/list를 그대로 return	routers/health.py, routers/data.py의 `status()` - FastAPI가 알아서 JSON으로 직렬화
`raise HTTPException(400, "메시지")`	routers/data.py: 컬럼 누락·행 수 부족 시

문법	이 프로젝트에서의 예시
잡지 못한 예외 → 500	Day1 lazy loading 실습에서 이미 다룬 것과 같은 원리

5. 앱 생명주기 & 정적 파일

문법	이 프로젝트에서의 예시
`@app.on_event("startup")`	main.py의 `startup()` - LOADING_MODE에 따라 lazy/eager 분기 (Day1 핵심)
`StaticFiles(directory=..., html=True)` + `app.mount("/", ...)`	main.py - 대시보드 HTML 서빙

> *라우터를 먼저 include_router()하고 StaticFiles를 마지막에 mount하는 이유: Starlette는 등록된 순서대로 경로를 검사하므로, 정적 파일을 먼저 mount하면 /predict 같은 API 경로도 정적 파일로 먼저 매칭되어 버립니다.*

6. 실행과 자동 문서

1. `uvicorn 모듈경로:app 변수 --reload` 형식으로 실행 - 예: `uvicorn serving_app.main:app --reload`
2. `/docs` (Swagger UI)에서 코드 한 줄 안 짜고 각 엔드포인트를 Try it out 으로 직접 호출·검증
3. 환경변수로 동작을 바꾸는 패턴 - 예: `MODEL_SOURCE=mlflow uvicorn ...`, `LOADING_MODE=eager uvicorn ...`

이 프로젝트에서 다루지 않는 것

아래 항목들은 이 프로젝트 코드 어디에도 등장하지 않으므로, 지금 몰라도 실습 진행에 지장이 없습니다.

- `Depends()` 의존성 주입
- 미들웨어(Middleware)
- 백그라운드 태스크(BackgroundTasks)
- WebSocket
- 인증/보안(OAuth2 등)
- Jinja2 템플릿
- 별도 `Response` 객체 직접 조작